

变量

Julia基础语法

苏州同元软控信息技术有限公司版权所有，
未经授权，不得复制、传播或以其他方式使用；
侵权必究。

苏州同元软控信息技术有限公司

2024年2月27日



TONGYUAN
Software and Control

Examples

第一个程序

```
julia> a = "Hello, World"  
"Hello, World"  
  
julia> print(a)  
Hello, World
```

苏州同元软控信息技术有限公司版权所有，侵权必究。

目录

1. 变量与常量

2. 数值类型

3. 数学运算

4. 复数和分数

5. 字符和字符串

6. 元组、字典与集合

1. 变量与常量

变量赋值: 与其他动态类型语言一样, Julia 无需提前声明变量类型, 可以直接创建

一般形式: `x = val`

```
julia> a = "Hello, World"  
"Hello, World"
```

```
julia> print(a)  
Hello, World
```

```
julia> b = 1  
1
```

```
julia> c = 2  
2
```

```
julia> d = b + c  
3
```

```
julia> x #x未定义  
ERROR: UndefVarError: x not defined
```

注意:

- 输入变量名则会显示该变量的值
- Julia中不会自动创建变量

1. 变量与常量

变量命名: Julia提供了非常灵活的命名策略。

```
julia> a = "Hello, World"
"Hello, World"

julia> a = 2
2

julia> 啊 = 10 #不推荐使用
10

julia> β = 36 #\beta,再敲击Tab键
36

julia> αp = 10 #先转义\alpha,在转义\^p
ERROR: syntax: "12" is not a valid function argument
name around REPL[6]:1

julia> 12a = 12 #不能以数字开头, 12a含义为12*a
ERROR: syntax: "12" is not a valid function argument
name around REPL[6]:1

julia> x,y,z = 11,12,13 #平行赋值法
(11, 12, 13)
```

变量名类型:

① 英文, 规则如下:

- 区分大小写;
- 不能以数字开头;
- 变量与函数名建议用下划线分隔;
- 类与模块首字母建议大写, 驼峰式;

② 中文: 不推荐使用

③ Unicode字符:

输入某个LaTeX符号 (比如\beta), 再敲击Tab键。

LaTeX符号详见: 《Julia 中文文档》- Unicode输入表

注意:

- 不能使用Julia中已有的单词的关键词作为变量名
- 同时定义多个常量时可以使用平行赋值法
- 部分LaTeX代码无法直接转义为Unicode

1. 变量与常量

常数声明: 用于声明特定、不变的量。

一般形式: `const x = val`

```
julia> const a = 1  
1
```

```
julia> const x,y,z = 11,12,13  
(11, 12, 13)
```

```
julia> a = 1 #未改变原值  
1
```

```
julia> a = 2 #修改常量会发出警告  
WARNING: redefinition of constant a. This may fail, cause  
incorrect answers, or produce other errors.
```

```
julia> a = 3.0 #修改常量时不允许修改数据类型  
ERROR: invalid redefinition of constant a
```

```
julia> a + 1  
3
```

注意:

- 常量新赋值不导致原常量值变化的情况下, 不会给出任何信息
- 常量新赋值的类型与原常量一致时, 会打印一个警告, 并替换为新的赋值
- 常量新赋值的类型与原常量不一致时, 会打印一个错误

1. 变量与常量

常量注意事项: **不推荐更改常量的值**，更改常量会引发多种问题或者非预期问题

```
julia> const x = 1
1

julia> f() = x
f (generic function with 1 method)

julia> f()
1

julia> x = 2
WARNING: redefinition of constant x. This may fail, cause
incorrect answers, or produce other errors.
2

julia> f() #f()在改变x值之前，已被编译
1.
```

说明:

如果一个方法引用了一个常量并且在常量被更改前已经被编译了，那么该函数很有可能继续使用旧值

目录

1. 变量与常量

2. 数值类型

3. 数学运算

4. 复数和分数

5. 字符和字符串

6. 元组、字典与集合

2. 数值类型

整数类型:

类型	是否带符号	比特数	最小值	最大值
Int8	√	8	-2^7	2^7-1
UInt8		8	0	2^8-1
Int16	√	16	-2^{15}	$2^{15}-1$
UInt16		16	0	$2^{16}-1$
Int32	√	32	-2^{31}	$2^{31}-1$
UInt32		32	0	$2^{32}-1$
Int64	√	64	-2^{63}	$2^{63}-1$
UInt64		64	0	$2^{64}-1$
Int128	√	128	-2^{127}	$2^{127}-1$
UInt128		128	0	$2^{128}-1$
Bool	N/A	8	false(0)	true(1)

```
julia> a = 1
1

julia> typeof(a) #操作系统为64位
Int64

julia> max = typemax(Int64)
9223372036854775807

julia> max + 1 #超限
-9223372036854775808

julia> min = typemin(Int64)
-9223372036854775808

julia> min - 1 #超限
9223372036854775807

julia> x = typeof(0x123)
UInt16

julia> Int64(0x123) #转化为Int64
291
```

说明:

- 根据操作系统不同, 整数 Int 可能是 Int32 或 Int64
- 超出一个类型可表示的范围会导致环绕
- 无符号整数会使用0x为前缀的十六进制来表示
- 可以使用T()进行数据类型转换



2. 数值类型

浮点类型： 用于表示小数

类型	精度	比特数
Float16	半(half)	16
Float32	单(single)	32
Float64	双(double)	64

```
julia> a = 5.2
5.2

julia> typeof(a) #操作系统为64位
Float64

julia> typeof(1e5) #操作系统为64位
Float64

julia> typeof(5.2f0) #使用则为32位
Float32

julia> Float32(a)
5.2f0
```

说明：

- 浮点数的默认类型取决于电脑系统是32位还是64位
- 浮点数可以用科学记数法表示
- 使用 “f” 替代 “e” 或 Float32()可以得到Float32的浮点数
- 半精度一般采用软件模拟，性能较差，不推荐使用



2. 数值类型

特殊浮点数：正零、负零、正无穷、负无穷、NaN

[illegible]

Float16	Float32	Float64	名称	描述
Inf16	Inf32	Inf	正无穷	大于所有有限浮点数的值
-Inf16	-Inf32	-Inf	负无穷	大于所有有限浮点数的值
NaN16	NaN32	NaN	非数	不等于任何浮点数，甚至不等于自己

```
julia> typemax(Float64)
Inf
```

```
julia> typemin(Float64)
-Inf
```

```
julia> 0/0
NaN
```

```
julia> Inf/Inf
NaN
```

```
julia> 0 * Inf
NaN
```

```
julia> NaN + 1 #传染性
NaN
```

```
julia> Inf + 1 #传染性
Inf
```

```
julia> NaN == NaN #NaN不等于任何值
false
```

```
julia> Inf == Inf
true
```

说明：

- Inf和NaN具有传染性
- NaN甚至不等于它自己

2. 数值类型

任意精度计算：BigInt和BigFloat分别提供了任意精度的整数和浮点数

```
julia> typemax{Int64} + 1 #超限
-9223372036854775808
```

```
julia> BigInt(typemax{Int64}) + 1 #扩位
9223372036854775808
```

```
julia> typeof(ans)
BigInt
```

[illegible]

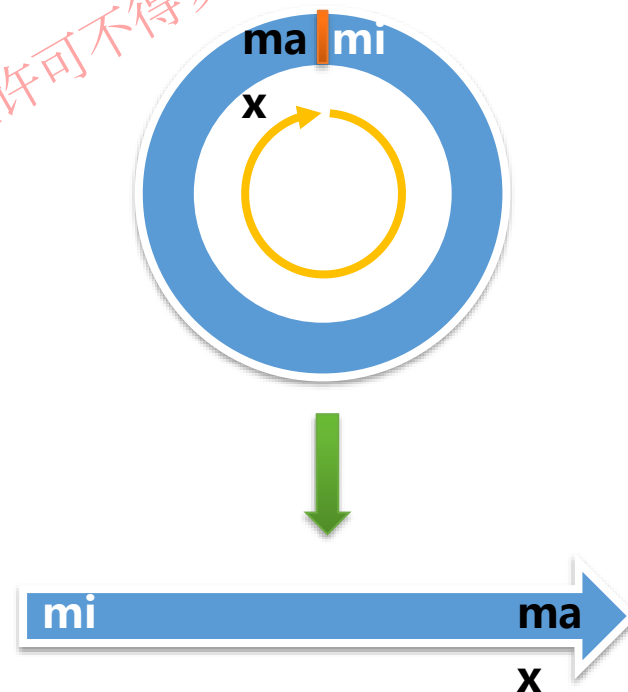
```
julia> 2.0^66/3
2.4595658764946067e19
```

[illegible]

```
julia> typeof(ans)
BigFloat
```

可使用以下语句对数值进行扩位：

- BigInt(x) 或 BigFloat(x)
- big""



目录

1. 变量与常量

2. 数值类型

3. 数学运算

4. 复数和分数

5. 字符和字符串

6. 元组、字典与集合

3. 数学运算

运算符:

类型	表达式	名称	描述
算数运算符	+x	一元加法	全等操作
	-x	一元减法	将值变为其相反数
	√x	平方根	求解平方根(\sqrt{x})
	x + y	二元加法	执行加法
	x - y	二元减法	执行减法
	x * y	乘法	执行乘法
	x / y	除法	执行除法
	x ÷ y	整除	取x/y的整数部分(\div)
	x \ y	反向除法	等价于y/x
	x ^ y	幂运算	x的y次幂
	x % y	取余	等价于rem(x,y)
布尔运算符	!x	非	取反计算
	x && y	与	取与计算
	x y	或	取或计算

```
julia> x = 1
1

julia> y = 2
2

julia> +x
1

julia> -x
-1

julia> √x
1.0

julia> x + y
3

julia> x - y
-1

julia> x * y
2
```

```
julia> x / y
0.5

julia> x \ y
2.0

julia> x ÷ y
0

julia> x ^ y
1

julia> x % y
1
```

```
julia> 2x
2

julia> 2(x+y)
6
```

```
julia> x = true
true

julia> y = false
false

julia> !x
false

julia> x && y
false

julia> x || y
true
```

说明:
直接放在标识符或括号前的数字, 如 2x 或 2(x+y) 会被视为乘法



3. 数学运算

运算符:

类型	表达式	名称	LaTeX代码
位运算符	$\sim x$	按位求反	
	$x \& y$	按位求与	
	$x y$	按位求或	
	$x \oplus y$	按位异或	$\backslash xor$
	$x \neg y$	非与	$\backslash nand$
	$x \vee y$	非或	$\backslash nor$
	$x \ggg y$	逻辑左移	
	$x \gg y$	算数右移	
	$x \ll y$	逻辑左移	
类型	列表		
复合赋值运算符	$+=$	$-=$	$*=$
	$/=$	$\div=$	$\div=$
	$\%=$	$\wedge=$	$\&=$
	$ =$	$\simeq=$	$\ggg=$
	$\gg=$	$\ll=$	

```
julia> x,y = Int8(-16), Int8(-10)
(-16, -10)

julia> bitstring(x), bitstring(y)
("11110000", "11110110")

julia> bitstring(~x)
"00001111"

julia> bitstring(x&y)
"11110000"

julia> bitstring(x | y)
"11110110"

julia> bitstring(x ⊕ y)
"00000110"

julia> bitstring(x ⊖ y)
"00001111"

julia> bitstring(x ∨ y)
"00001001"
```

```
julia> bitstring(x >>> 2)
"00111100"

julia> bitstring(x >> 2)
"11111100"

julia> bitstring(x << 2)
"11000000"

julia> x = 1
1

julia> x += 3
4

julia> x
4
```

说明:
x += 3 等价于 x = x + 3



3. 数学运算

运算符:

类型	表达式	含义
数值比较运算符	$x == y$	相等
	$x != y,$ $x \neq y$	不等于
	$x < y$	小于
	$x \leq y,$ $x \leq y$	小于等于
	$x > y$	大于
	$x \geq y,$ $x \geq y$	大于等于

```
julia> 1 == 1
true

julia> 1 == 2
false

julia> NaN == NaN
false

julia> isequal(NaN,NaN)
true

julia> 0.0 === -0.0 #二进制表现形式不同
false

julia> bitstring(x ⊘ y)
"00000110"

julia> bitstring(x ⊗ y)
"00001111"

julia> bitstring(x ⊖ y)
"00001001"
```

苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
侵权必究。



3. 数学运算

运算符优先级:

分类	运算符	结合性
幂运算	$\wedge$	右结合
一元运算符	$+ - \sqrt{}$	又结合
移位运算符	$<< >> <<<$	左结合
乘、除法和按位与	$* / \% \& \backslash \div$	左结合
加、减法、按位或和按位异或	$+ - \vee$	左结合
比较操作符	$> < >= <= == == != != == <:$	无结合性
赋值运算符和更新运算	$= += -= *= /= // = \backslash = \wedge = \div = \% = = \& = \times = < < = > > = > > > =$	右结合

高

低

```
julia> x = 3; 2x^2
18

julia> x = 3; 2^2x
64

julia> x = 3; -x^2 #等价于-(x^2)
-9

julia> x = 3; x^-2 #等价于x^(-2)
0.11111111111111111

julia> x = 3; x^(-2)
0.11111111111111111
```

建议:
复合运算时使用()进行区分优先级



目录

1. 变量与常量

2. 数值类型

3. 数学运算

4. 复数和分数

5. 字符和字符串

6. 元组、字典与集合

4. 复数和分数

复数的定义

一般形式: $a + b \cdot \text{im}$

Julia中复数的虚部用“im”表示, im为全局变量。

注意:

不能使用i和j来表示虚部

复数的构建

方法一(im直接构建):

```
Julia> 1 + 2im  
1 + 2im
```

```
julia> 1+Inf*im  
1.0 + Inf*im
```

```
julia> 1+NaN*im  
1.0 + NaN*im
```

注意:

$a + b \cdot \text{im}$, b如果为变量名称时则错误

方法二(complex函数构建):

```
julia> a=1;b=2;complex(a,b)  
1 + 2im
```

4. 复数和分数

复数的运算

```
julia> 1+2im
1 + 2im

julia> (1+2im)*(2-3im) #复数乘法
8 + 1im

julia> (1+2im)/(1-2im) #复数除法
-0.6 + 0.8im

julia> (1+2im)+(1-2im) #复数加法
2 + 0im

julia> (1+2im)+0.5 #复数和实数可以直接运算
1.5 + 2.0im

julia> (-1+2im)^2.5 #复数指数运算
2.729624464784009-6.9606644595719im

julia> 2im^2 #复数求解的结果还是复数
-2+0im

julia> 1+3/4im #4im的优先级最高
1.0 - 0.75im
```

说明:

- 不同数据类型可直接进行计算
- 复数运算的结果必为复数
- 系数的优先级比除法的优先级更高:

$$3/4im == 3/(4*im) == -(3/4) im$$

4. 复数和分数

复数的初等函数使用

关键字	描述
real	实部
imag	虚部
conj	复共轭
abs	绝对值
abs2	取平方后的绝对值
angle	以弧度为单位的相位角
sqrt	开根号
cos	求余弦
exp	指数运算
sinh	双曲正弦函数运算

```
julia> real(1+2im) #取实部
1

julia> imag(1+2im) #取虚部
2

julia> conj(1+2im) #求复共轭
1 - 2im

julia> abs(1+2im) #求绝对值
2.23606797749979

julia> abs2(1+2im) #取绝对值的平方
5

julia> angle(1+2im) #取相位角
1.1071487177940904

julia> cos(1+2im)
2.0327230070196656 - 3.0518977991517997im

julia> exp(1+2im)
-1.1312043837568135 + 2.4717266720048188im
```

说明:

- 复数的绝对值(abs)是从零点到它的距离
- abs2给出绝对值的平方
- 所有其他的初等函数在复数上都可使用

```
julia> sqrt(1im)
0.7071067811865476 + 0.7071067811865475im

julia> sqrt(1+2im)
1.272019649514069 + 0.7861513777574233im

julia> sqrt(-1) #负数不能直接开方，要改为复数形式
ERROR: DomainError with -1.0:
sqrt will only return a complex result if called with a
complex argument. Try sqrt(Complex(x))

julia> sqrt(-1+0im)
0.0 + 1.0im
```

注意:

虽然-1 == -1 + 0im，但是要对负数求平方根，负数只能写成复数的形式。



4. 复数和分数

分数的构建

一般形式: `a // b`

分数通过 “//” 构建, 用于表示整数精确比值的分数类型

```
julia> 2//3  
2//3
```

注意:
分子分母只能为**整型**。

分数的标准化

如果一个分数的分子和分母含有公因子, 它们会被约分到最简形式且分母非负:

```
julia> 6//9  
2//3  
  
julia> -4//8  
-1//2  
  
julia> 5//-15  
-1//3  
  
julia> -4//-12  
1//3
```

分数的标准化分子和分母分别可以使用 `numerator` 和 `denominator` 函数得到:

```
julia> numerator(4 // 6) #查看标准化分子  
2  
  
julia> numerator(2//3)  
2  
  
julia> denominator(4//6) #查看标准化分母  
3  
  
julia> denominator(2//3)  
3
```

4. 复数和分数

分数的运算: “//” 优先级高于所有运算符(除im以外)

```
julia> 2//3 == 6//9  
true
```

```
julia> 3//7 < 1//2  
true
```

```
julia> 2//4 + 1//6  
2//3
```

```
julia> 5//8 * 3//12  
5//32
```

```
julia> 2//7*(1+2im) #2//7优先级最高  
2//7 + 4//7*im
```

```
julia> 1//2+2im #1//2优先级最高  
1//2 + 2//1*im
```

```
julia> 1+2//3im #3im优先级最高  
1//1 - 2//3*im
```

```
julia> float(1//2) #分数转化为小数  
0.5
```

```
julia> a=1;b=2;
```

```
julia> isequal(float(a//b),a/b)  
true
```

```
julia> 5//0  
1//0
```

```
julia> -3//0  
-1//0
```

```
julia> float(ans)  
Inf
```

```
julia> 0//0  
ERROR: ArgumentError: invalid rational:  
zero(Int64)//zero(Int64)
```

- 使用float () 将值转换为合适的浮点数类型,且完全相等
- 除a==0且b==0时, 任意整数值a,b从分数到浮点数转换遵循数值一致性
- Julia接受构建无穷分数值, 但不接受构建NaN分数值

目录

1. 变量与常量

2. 数值类型

3. 数学运算

4. 复数和分数

5. 字符和字符串

6. 元组、字典与集合

5. 字符与字符串

字符与字符串声明

Julia中, Char表示单个字符, 用单引号包围
String表示字符串, 用双引号或三引号包围
Char不等于String

```
julia> 'x' #Char类型
'x': ASCII/Unicode U+0078 (category Ll: Letter, lowercase)

julia> typeof('x')
Char

julia> "中" #String类型
"中"

julia> typeof("中")
String

julia> print("""中国:"欢迎你"。""")
中国:"欢迎你"。 #字符串中使用引号, 最外侧需要使用"""

julia> typeof(ans)
String

julia> 'x' == "x" #不同类型
false
```

字符与字符串运算: 基于Unicode代码进行运算

```
julia> 'A' < 'a' #Char值比较, Int64('A')=65, Int64('a')=97
true

julia> 'x' - 'a' #Char值有限运算, Int64('A')=120, Int64('a')=97
23

julia> 'A' + 1 #求解结果还是Char
'B': ASCII/Unicode U+0042 (category Lu: Letter, uppercase)

julia> s = "\u2200 x \u2203 y" #\u2200可以根据Unicode代码直接转义
"∀ x ∃ y"
```

5. 字符与字符串

字符串索引

函数	描述
firstindex()	第一个元素的索引值
lastindex()	最后一个元素的索引值
length()	字符串的字符数

关键字	描述
begin	第一个元素
end	最后一个元素

说明:

- 字符串的索引是从1开始的
- 可以使用end及begin进行运算
- begin表示为1，end表示为字符串长度
- end÷2结果不为整数时，采用去尾法取证
- 下标小于开头begin (1) 或者大于结尾end 都会导致错误
- 字符串的索引可使用范围来索引
- str[k] 输出结果为字符，str[k:k]输出结果为字符串

```
julia> str= "Hello,world.\n"; str=[begin] #索引初始字符
'H': ASCII/Unicode U+0048 (category Lu: Letter, uppercase)

julia> str[1] #第1个Char, 结果为Char
'H': ASCII/Unicode U+0048 (category Lu: Letter, uppercase)

julia> str[end-1] #倒数第2个Char
',': ASCII/Unicode U+002C (category Po: Punctuation, other)

julia> str[end÷2] #第(length/2)个Char
',': ASCII/Unicode U+002C (category Po: Punctuation,other)

julia> str[begin-1] #第0个Char, 从1开始索引, 不存在第0个
ERROR: BoundsError: attempt to access 13-codeunit String at index [0]

julia> str[end+1] #第(end+1)个Char, 超限
ERROR: BoundsError: attempt to access 13-codeunit String at index [14]

julia> str[4:9] #第4到9个Char, 结果为String
"lo,wor"

julia> str[6] #第6个Char, 结果为Char
',': ASCII/Unicode U+002C (category Po: Punctuation, other)

julia> str[6:6] #第6个String, 结果为String
","
```



5. 字符与字符串

分割与拼接

函数	描述
split(x)	在出现分隔符 dlm 时将 str 拆分为子字符串数组
join(x,y)	拼接，合并字符串
string(x,y)	拼接字符串
*	拼接字符串
^	重复

说明：

- string(), “*” 均可用于字符和字符串拼接；
- join只适用于字符串的拼接；

```
julia> split("AB CD") #以空格分隔
"AB"
"CD"

julia> join("ab", "cd") #字符串组合
"abcd"

julia> join('a', 'c') #join不适用字符拼接
"a"

julia> string("Hello", " ", "world") #多个字符串组合
"Hello,world"

julia> string('a', 'c') #字符组合为字符串
"ac"

julia> "Hello" * " ", " * "world" #多个字符串组合
"Hello,world"

julia> 'a' * 'c' #字符组合为字符串
"ac"

julia> "AB"^3 #多个字符串组合
"ABABAB"
```

苏州同元软控信息技术有限公司版权所有，侵权必究。
未经许可不得复制、传播或以其他方式使用；



5. 字符与字符串

其他应用

函数	描述
lowercase(x)	转化为小写
uppercase(x)	转化为大写
replace()	字符替换
startswith()	判断开始字符串
occursin()	在字符串中某子字符串可否找到
strip()	去除开始空格

```
julia> lowercase("ABCD") #小写
"abcd"

julia> uppercase("abcd") #大写
"ABCD"

julia> replace("ABCD", 'A'=>'E') #替换
"EBCD"

julia> startswith("julia is interesting", "julia")
true

julia> occursin("julia", "julia is interesting")
true

julia> startswith(strip("julia is interesting"), "julia")
true
```

关键字	描述
\	多行字符串合并为一行 特殊字符显示
\$	插入值于字符串

```
julia> "This is a long \
line" #反斜杠划分长字符串
"This is a long line"

julia> print("I have \$100 in my account.\n")
I have $100 in my account.

julia> "1 + 2 = $(1 + 2)" #$()用于向字符串中插值
"1 + 2 = 3"

julia> a=3; "1 + 2 = $a"
"1 + 2 = 3"
```

目录

1. 变量与常量

2. 数值类型

3. 数学运算

4. 复数和分数

5. 字符和字符串

6. 元组、字典与集合

6. 元组、字典与集合

元组(Tuple):与数组类似都是有序的元素集合，可以容纳**任何值**，且元素**不能更改**。

元组通过圆括号和逗号来构造，其内容可以通过索引来访问

一般形式: (...)

```
julia> x1=(1,2,3,4) #创建一维元组
(1, 2, 3, 4)
```

```
julia> x1[1] #访问第1个元素
1
```

```
julia> length(x1) #元素个数
4
```

```
julia> x2=(1,2.4,'a', "hello") #各个元素类型可以不同
(1, 2.4, 'a', "hello")
```

```
julia> x3 = (a = 1, b = 2) #键值同在一个元组内
(a = 1, b = 2)
```

```
julia> x3.a #访问元组
1
```

```
julia> x,y = 2,4 #键值便捷创建形式
(2, 4)
```

```
julia> x#访问元组
2
```

```
julia> x1[1] = 3 #不能更改元组的值
ERROR: MethodError: no method matching setindex!
(::NTuple{4, Int64}, ::Int64, ::Int64)
Stacktrace:
 [1] top-level scope
      @ REPL[15]:1
```

```
julia> x4=((1,2),(3,4)) #创建二维元组
((1, 2), (3, 4))
```

```
julia> x4[1]#访问二维元组元素，输出第一维元素
(1, 2)
```

```
julia> x4[1][2]#访问二维元组元素，输出第一维元素
2
```

6. 元组、字典与集合

元组使用： 作为函数参数

一般形式： `y=f(...)`或`y=f(...;...)`

```
julia> function f(a,b) #定义函数
    a
end
```

```
julia> y1 = f(1,2) #方法1
1
```

```
julia> input1=(a=2,b=4)
(a = 2, b = 4)
```

```
julia> y2=f(input1...) #方法2, 使用键值对需要用...
2
```

```
julia> input2=(b=2,a=4)
(b = 2, a = 4)
```

```
julia> y2=f(input2...) #依旧按照位置传参
2
```

```
julia> function f(;a,b) #定义函数
    a
end
```

```
julia> input3=(b=2,a=4)
(b = 2, a = 4)
```

```
julia> y3=f(;input3...) #按照参数名传参
4
```

```
julia> a,b = 2,4
(2, 4)
```

```
julia> f(;a, b)
2
```

6. 元组、字典与集合

字典(Dict): 字典是一个映射也属于一种容器，容纳的是一个个**键值对**，且是**无序**的。

区别于数组，字典的索引几乎可以是**任何类型**

一般形式: Dict(key1 => value1, key2 => value2,...)

```
julia> dic1 = Dict{"aa"=>1,"bb"=>2,"cc"=>3} #无序的,方式一
Dict{String, Int64} with 3 entries:
  "bb" => 2
  "aa" => 1
  "cc" => 3
```

```
julia> dic2 = Dict{1=>1,2=>2,3=>3}; #key不一定是字符串
```

```
julia> dic3 = Dict{((1,1)=>2,(1,2)=>3)}; #key不一定是字符串
```

```
julia> length(dic1) #键值对数量
3
```

```
julia> keys(dic1) #获取键
(获取字典的键)
```

```
julia> values(dic1) #获取值
(获取字典的值)
```

```
julia> dic1["aa"] #值调用
1
```

```
julia> ary=["one"=>1,"two"=>2] #定义数组
2-element Vector{Pair{String, Int64}}:
 "one" => 1
 "two" => 2
```

```
julia> Dict(ary) #数组转化为字典
Dict{String, Int64} with 2 entries:
  "two" => 2
  "one" => 1
```

```
julia> tpl= [("one",1), ("two",2)]
2-element Vector{Tuple{String, Int64}}:
 ("one", 1)
 ("two", 2)
```

```
julia> Dict(tpl)
Dict{String, Int64} with 2 entries:
  "two" => 2
  "one" => 1
```

说明:
数组可转化为字典

6. 元组、字典与集合

集合(Set): 容纳**同类型**、**互不相等** (即**唯一**) 的元素, 且元素是**无序**的。

```
julia> S=Set((1,2,3,2,3)) #无序的
```

```
Set{Int64} with 3 elements:
```

```
2
```

```
3
```

```
1
```

```
julia> S=Set(1:3:10) #无序的
```

```
Set{Int64} with 4 elements:
```

```
4
```

```
7
```

```
10
```

```
1
```

```
julia> 2 in S #判断2是否在S中
```

```
false
```

```
julia> A=Set((1,2,3,4)); issubset((1,2),A)
```

```
true
```

```
julia> 2 in A
```

```
true
```

说明: `issubset()`用于判断子集或元素是否在集合内
`in` 用于判断元素是否在集合内

```
julia> A=Set((1,2,3,4));B=Set((2,4,6,7));intersect(A,B) #交集
```

```
Set{Int64} with 2 elements:
```

```
4
```

```
2
```

```
julia> setdiff(A,B) #差集
```

```
Set{Int64} with 2 elements:
```

```
3
```

```
1
```

```
julia> union(A,B) #并集
```

```
Set{Int64} with 6 elements:
```

```
4
```

```
6
```

```
7
```

```
2
```

```
3
```

```
1
```

说明: `intersect()`用于求交集
`setdiff()`用于求差集
`union()`用于求并集

建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

请各位专家指正！

苏州同元软控信息科技有限公司版权所有，侵权必究。
传播或以其他方式使用；